

آیا کد هنوز هم مهمه؟ جواب اینه که بله کد همیشه مهم بوده. برای مثال شرکت‌هایی تو طول تاریخ بودن که به خاطر کد اشتباه، نابود شدن.

هزینه نگهداری یه کد بد نوشته شده، به مرور زمان بیشتر و بیشتر میشه تا حدی که دیگه Productivity رو به صفر می‌رسونه.

وقتی کدی بد نوشته میشه، نگهداریش سخت میشه و باعث میشه آدم‌ها ترجیح بدن برن سر یه پروژه دیگه و از صفر شروع کنن، یا همین پروژه رو دوباره از اول بنویسن، اتفاقی که خب نمیتونه بیوفته، پس بهتره کد از اول درست نوشته بشه.

مفهوم Rigidity:

سفتی به معنی گرایش نرم‌افزار به سختی و مقاومت در برابر تغییره. کدی که بد نوشته شده باشه، در برابر تغییر، هر چقدر هم که کوچیک باشه، مقاومت میکنه و هر تغییری باعث یه ایجاد اتفاق‌های ناخواسته تو قسمت‌های وابسته دیگه میشه.

مفهوم Fragility:

شکنندگی به معنی گرایش نرم‌افزار به خراب شدن تو بخش‌های مختلف، بعد از ایجاد تغییره. این خرابی‌ها تو نقاطی که ربط مستقیمی و مفهومی‌ای با بخش تغییر داده شده ندارن، معمولاً اتفاق میوفتن.

مفهوم Immobility:

لختگی به معنی عدم وجود قابلیت استفاده منابع استفاده شده در پروژه‌های گذشته، تو پروژه‌های جدید. البته این حالت تو یه پروژه هم میتونه پیش بیاد، یعنی مثلاً کدی که چند روز پیش نوشته شده، الان دیگه نمیتونه تو بخش جدید پروژه استفاده بشه. این اتفاق معمولاً وقتی میوفته که فردی از تیم متوجه میشه به ویژگی‌ای نیاز داره که فرد دیگه‌ای تو تیم، قبلاً روش کار کرده.

مفهوم Opacity:

شفافیت کد به معنی وضوح کد برای دیگر افراد تیمه. هر چقدر هم تلاش کنیم، کد نمیتونه برای تمام افراد کاملاً خوانا باشه ولی خب تلاش ما اینه بیشترین خوانایی رو داشته باشیم.

چرا ما بد کد می‌نویسیم؟ با اینکه میتونیم یه سری بهونه بیاریم، مثل اینکه مدیر پروژه فلان بود فلان بیسار بود، ولی جواب کوتاه اینه که، ما حرفه‌ای نیستیم.

خب تا الان توجیه شدیم که چرا نیازه تا کد تمیز بنویسیم. سوال اینجاست که clean code اصن چی هست؟ به چه کدی می‌گیم یه clean code؟

اسطوره‌های برنامه‌نویسی تو طول تاریخ نظراتی داشتن در این مورد.

- کدی خوبه که "یک" کار رو خوب انجام بده.
- کد تمیز، ساده و مستقیمه و مثل یه متن ادبی روون، راحت خونده میشه.
- کد تمیز، کدیه که معلومه توسط کسی که واقعا داستان براش مهمه نوشته شده.
- کدی که کد تمیزه، طبق انتظارات پیش میره.

Clean code چیه؟

قوانین و چهارچوب‌هایی که به دولوپرا کمک میکنه کدی بنویسن که راحت خونده بشه، راحت میتونه نگهداری بشه و راحت هم توسعه پیدا کنه. هدف این قوانین اینه که قابلیت خوانایی و نگهداری کد رو افزایش بدن و در نهایت اونو کارآمدتر و کم‌ارورتر بکنن.

اجزای Clean Code Principle:

- 1- قراردادهای نام‌گذاری (Naming Conventions)
- 2- تکرار ممنوع (DRY (Don't Repeat Yourself)
- 3- اصل تک مسئولیتی (SRP (Single Responsibility Principle)
- 4- جداسازی نقاط پراهمیت (Separation of Concerns)
- 5- کامنت‌ها و داکیومننت‌نویسی (Comments and Documentation)
- 6- ساده نگهش دار ابله (KISS (Keep It Simple Stupid)
- 7- قرار نیست بهش احتیاج پیدا کنی (YANGI Principle (You are not gonna need it)
- 8- تست‌نویسی و قابلیت تست (Testing and Testability)
- 9- ریفکتور کردن (Refactoring)

قراردادهای نام‌گذاری:

باید از اسم‌های بامعنی و گویا برای متغیرها، توابع و کلاس‌ها استفاده کنیم. اسم باید هدف و کارایی رو کاملا مشخص کنن. خصوصا باید از اسم‌های مخفف شده، مبهم و یا تک حرفی پرهیز کنیم مگه اینکه کاملا هدف اون متغیر یا تابع تو ناحیه مسئله مشخص باشه.

تکرار ممنوع:

باید از تکرار کردن و کپی پیست کردن کد دوری کنیم. به جاش باید از کلاس‌ها و توابع یا ماژول‌ها استفاده کنیم تا بتونیم دوباره از کد استفاده کنیم.

اصل تک مسئولیتی:

هر کلاس یا ماژول دقیقا باید یک مسئولیت یا دلیل برای تغییر دادن داشته باشه. اگه یه تابع یا کلاس، بیش از یک مسئولیت داره و بیش از یک کار رو انجام میده، بهتره که اونو به بخش‌های کوچیکتر ریفکتور کنیم.

جداسازی نقاط پراهمیت:

بخش های مختلف سیستم، مثل منطق و داده و نمایش، باید از هم جدا بشن. باید تو ماژول ها و لایه های مختلف نگه داری بشن. این باعث میشه که قابلیت نگه داری محصول بالا بره و ایجاد تغییر تو یک بخش، حداقل تاثیر روی بخش های دیگه رو داشته باشه.

کامنت گذاری و داکيومنت نویسی:

برای منطق های پیچیده، الگوریتم های پیچیده و یا هر کد غیر واضحی، کامنت می نویسیم، ولی نکته ای که وجود داره اینه که باید تا جایی که میتونیم کد خود-توضیح-دهنده ای بنویسیم. نباید جوری کد بنویسیم که وجود کامنت های ضروری باشن، کامنت ها صرفا باید کمک کننده باشن.

ساده نگهش دار ابله:

باید کدی که مینویسیم، تا جایی که ممکنه ساده و قابل فهم باشه. باید حواسمون باشه الکی نیایم بدون دلیل کد رو پیچیده کنیم. باید کارها رو به ساده ترین نحوه ممکن انجام بدیم. وجود پیچیدگی تو کد، باید برای حل مسئله باشه، نه برای اینکه کسش فقط یه چیزی رو چون میتونیم، پیچیده بنویسیم.

تست نویسی و قابلیت تست:

برای هر بخش کد باید تست نوشته بشه تا از صحت عملکرد کد مطمئن بشیم. باید با استفاده از تکنیک هایی مثل Mocking و Dependency Injection، کد قابل تست بنویسیم.

ریفکتور کردن:

برای اینه ساختار، قابلیت خوانایی و قابلیت نگه داری کد بالاتر بره، باید هر چند وقت یه بار برگردیم و کدهایی که نوشتیم رو ریفکتور کنیم. ریفکتور کردن باید تو قدم های کوچیک و یواش یواش اتفاق بیوفته تا نتیجه تست های کد خراب نشن و کد از کار نیوفته.

نام گذاری زیر ذره بین:

اسم باید قصد و نیت ما از ساخت یه متغیر یا تابع رو کاملا مشخص کنه. اسم باید جوری باشه که از سوتفاهم جلوگیری کنه و این داستانا. اسم باید قابل تلفظ هم باشه، راحت بچرخه تو ذهن. باید همیشه تو کد، از یه نوشتن استفاده کنیم تا خوندن کد برامون راحت تر بشه. نه که یه جا PascalCase یه جا camelCase یه جا ALLCAPS و اینا. یه نکته دیگه هم درمورد نام گذاری، اینه که بهتره برای اسکوپ های کوچیک، از اسم های کوچیک هم استفاده کنیم. طبیعتا برای اسکوپ های بزرگ هم از اسم های طولانی تر و توضیحی تر.

هر کسی میتونه کدی بنویسه که ماشین بفهمتش، برنامه نویس خوب اونیه که بتونه کدی بنویسه که آدما راحت بفهمنش.

توابع:

وقتی کد رو تحویل میدیم، انگار یه قرضی گرفتیم، قرضی که با نوشتن دوباره کد و ترو تمیز کردنش داریم پسش میدیم. هر چی بیشتر میگذره، اگه دوباره به کد برنگردیم، این قرض همینجور سنگین و سنگین تر میشه و به فنا میریم.

هر چی از زمان پروژه بگذره، تغییر دادن توش سخت تر میشه، فرقی نداره کدی که نوشتیم refactor شده باشه یا نه، این اتفاق میوفته. فرقیشون تو اینه که کد refactor شده باز خیلی آسون تر و با هزینه خیلی کمتری میتونه تغییر پیدا کنه.

خب پس ما میایم از توابع استفاده میکنیم تا هم پروژه راحت تر توسعه پیدا کنه هم نگه داریش آسون تر باشه.

اولین قانونی که تو استفاده از توابع باید یادمون باشه، اینه که تا جایی که ممکنه، بدنه تابع باید کوچیک باشه. قدیم میگفتن فلان قدر خط باشه که تو یه مانیتور جا شه، الان اون معیار دیگه وجود نداره ولی خب تا جایی که میشه باید کوچیک باشه.

کلا if یا else یا while هایی که مینویسیم، باید تا حد امکان تک خطی باشن تا خوانایی کد بالا بره. نام گذاری درست هم باعث میشه خوانایی کد بالا بره و فیلان بیسار.

توابع هم باید دقیقاً یک کار رو انجام بدن. دقیقاً یک کد کار و اون کار رو هم باید درست انجام بدن. مسئولیت یک کار از سیستم باید بر عهده یک تابع باشه.

تو نام گذاری توابع و به طور کلی تو نام گذاری، نباید از استفاده از اسم های بزرگ بترسیم. استفاده از یه اسم بزرگ که هدف تابع رو توضیح میده، به مراتب بهتر از نوشتن یه کامنته توضیحیه. در نهایت، از اینکه انتخاب نام ازمون وقت بگیره هم نباید بترسیم.

ورودی توابع:

مقدار ایده آل برای ورودی های یه تابع، صفره (که بهش Niladic هم میگیم). بعد از اون، بهتره که تابع فقط یک ورودی داشته باشه (Monadic). خیلی ضروری بود دیگه باشه تابع میتونه دو تا ورودی هم داشته باشه (Dyadic) ولی خب دیگه از سه تا ورودی (Triadic) بیشتر نباید بشه به هیچ وجه. خود 3 ورودی هم باید ازش دوری بشه.

کلا آرگومان ها، کار رو سخت میکنن و فهم کد رو سخت تر میکنن. مخصوصاً برای تست کردن کد، وجود آرگومان ها کار رو سخت میکنن. درک آرگومان های خروجی از درک آرگومان های ورودی سخت تر هم هست. معمولاً میگیم که اطلاعات از طریق آرگومان ها وارد تابع میشن و به صورت return value ازش خارج میشن. اینکه میگیم آرگومان ها درک کد رو سخت میکنن یعنی چی؟ کد پائین رو ببین

```
Argument = 15;  
TestArgument(Argument);  
Debug.log(Argument); ?
```

خط سوم میخواد مقدار Argument رو چاپ کنه. تو خط دوم، Argument به عنوان پارامتر به تابع پاس داده شده. نکته اینجاست که ما الان نمیدونیم چه چیزی قراره چاپ شه.

انواع آرگومان‌ها:

Flag Arguments

یه نوع از آرگومان‌هایی که باهاش ممکنه سرو کار داشته باشیم، Flag Argument ها هستن. وقتی تابع Flag argument داره، با مقدار True یه کار انجام میده و با مقدار False یه کار دیگه.

Argument Objects

یه نوع دیگه، Argument Object ها هستن. به جای اینکه متغیرها رو دونه دونه به عنوان آرگومان به تابع بدیم، میتونیم یه آبجکت شامل چند تا متغیر بدیم بهش. اینجوری تعداد آرگومان‌های یه تابع رو هم کاهش دادیم.

Argument Lists

یه نوع دیگه، لیست آرگومان‌هائ. وقتی از لیست آرگومان‌ها استفاده می‌کنیم، تمام عناصر لیست دونه به دونه به تابع داده میشن و عملیات روشن انجام میشه.

به طور کلی، توابع باید یا یک کار رو انجام بدن، یا یک چیزی رو جواب بدن.

Unit Test – Refactoring – Profiling

چطوری داستان کار میکنه اصن؟ کجا از اینا استفاده میشه؟

وقتی یه کدی مینویسیم یا یه محصول نرم‌افزاری درست میکنیم، اول از همه باید مطمئن بشیم کد داره درست کار میکنه. چجوری مطمئن میشیم کد داره درست کار میکنه؟ با استفاده از Unit Test

بعد از اون، باید مطمئن بشیم که کد خوانایی بالایی داره و حداقل پیچیدگی رو داره. چجوری از این مطمئن میشیم؟ با استفاده از Refactoring

بعد از این دو مرحله، باید performance محصول رو در صورت نیاز، بهبود بدیم. این کار رو چجوری انجام میدیم؟ با استفاده از Profiling

Testing

کلا مفهوم تست، به چند بخش تقسیم میشه و ما تست رو تو ابعاد مختلفی داریم.

:Unit Testing

یعنی هر جز به تنهایی مورد بررسی قرار میگیره و صحت کاراییش تأیید میشه.

:Module Testing

یه مجموعه از اجزای مرتبط بهم، با همدیگه تست میشن.

:Sub-System Testing

عدم تطابق رابط‌های زیر سیستم‌ها با هم، اینجا بررسی میشن.

:System Testing

رابطه بین زیر سیستم‌ها با همدیگه اینجا بررسی میشه. علاوه بر اون، بررسی میشه که آیا سیستم نهایی میتونه نیاز اصلی رو برطرف کنه یا نه.

:Acceptance Testing

تو این مرحله، سیستم رو با ورودی دیتاهای واقعی تست میکنیم. تا قبل همه دیتاها شبیه‌سازی شده بودن.

اصن یونیت تست چی هست؟

قدیما، قبل از اینکه از مفهوم یونیت تست استفاده بشه، سیستم‌ها به عنوان یک کل مورد بررسی قرار میگرفتن. این باعث میشد یه سری از اجزای سیستم اصن مورد بررسی قرار نگیرن، مشخص نباشه که ارورها دقیقا تو کدوم بخش دارن اتفاق میوفتن و پیگیری مشکل کار خیلی سختی باشه.

بعد از اون، مفهوم یونیت تست تعریف شد. تو یونیت تستینگ، هر جز به صورت جداگونه مورد بررسی قرار میگیره و تک تک اجزا حداقل یک بار تست میشن. این باعث میشه ارورها (در صورت وجود)، صد در صد و زودتر از روش تست قبلی، بروز داده بشن. چون محدوده ارورها کوچیکتره، حل اون ارورها هم آسونتر میشه اینجوری.

یونیت تست باید چه ویژگی‌هایی داشته باشه؟

قابل ایزوله‌سازی باشه (دقیقا به تنهایی بتونه انجام بشه)

قابل تکرار باشه

قابل اتوماتیک‌سازی باشه

نوشتنش راحت باشه

چرا از یونیت تست استفاده کنیم اصن؟

دییگ کردن کد راحت‌تر میشه

باعث میشه به محض بروز مشکل، متوجه شیم و برگردیم عقب

هزینه‌های پروژه رو در آینده کاهش میده

روند توسعه رو سریعتر می‌کنه

طراحی بهتری رو به محصول میاره

ریفکتور کردن

اصن ریفکتور کردن چیه؟

دو تا تعریف برای ریفکتور کردن وجود داره.

- 1) به پروسه تغییر دادن محصول، طوری که رفتارهای کد عوض نشه و همون کار قبلی رو انجام بده و همزمان ساختار داخلی کد بهتر بشه، میگیم ریفکتور کردن.
- 2) تغییری که تو کد میدیم، بدون اینکه به رفتارش دست زده باشیم و فقط معیارهای غیر رفتاری مثل سادگی، انعطاف پذیری و فهم ساده‌تر رو ارتقا داده باشیم، میشه ریفکتور کردن.

چرا باید ریفکتور کنیم اصن؟

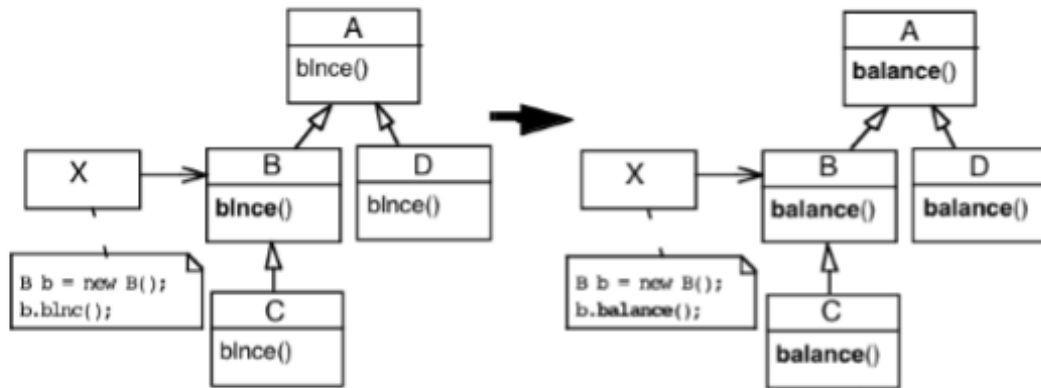
چند تا دلیل وجود داره. تو پروژه، اینکه ما همون اول بتونیم به بهترین طراحی فکر کنیم و از همون اول خشت اول رو بر اساس بهترین طراحی ممکن بزاریم، تقریباً غیر ممکنه. جدا از اون، اینکه از همون اول دقیق مسئله رو متوجه بشیم، دقیق نیازهای کاربر رو درک کنیم و اینا هم ممکن نیست. در ضمن، اینکه از اول به نقشه ذهنی کامل و بدون نقص از اینکه پروژه چطور قراره رشد کنه و بزرگ بشه داشته باشیم هم غیر ممکنه. کلاً تو پروژه‌ها، طراحی‌های اولیه کامل و کافی نیستن. هر کاری هم که بکنیم، با گذر زمان، سیستم‌ها سفت‌تر میشن و تغییر دادنشون سخت میشه. پس به ریفکتور کردن نیاز داریم.

چند نمونه از کارهایی که برای ریفکتور کردن میتونیم انجام بدیم اینان:

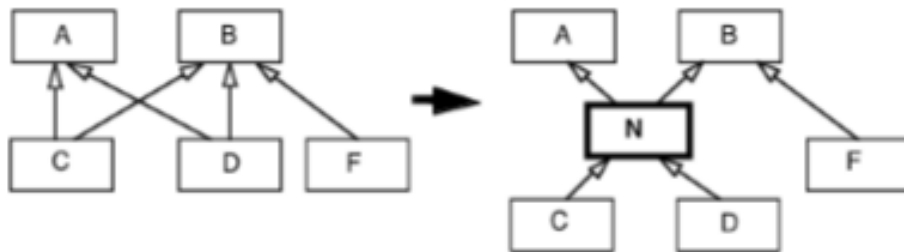
Add Parameter
Change Association
Change Reference to Value
Change Value to Reference
Collapse Hierarchy
Consolidate Conditional
Convert Procedures to Objects
Decompose Conditional
Encapsulate Collection
Encapsulate Downcast
Encapsulate Field
Extract Class
Extract Interface

Extract Method
Extract Subclass
Extract Superclass
Form Template Method
Hide Delegate
Hide Method
Inline Class
Inline Temp
Introduce Assertion
Introduce Explain Variable
Introduce Foreign Method
...

عوض کردن اسم توابع:



اضافه کردن کلاس‌ها (یه لایه جدید):



ساده‌سازی سلسله مراتب:

وقتی یه کلاس پدر و یه کلاس فرزند خیلی با هم فرقی ندارن، یکی شون میکنیم.



ادغام شرطها:

وقتی به تیکه کد چند جا تکرار شده، میتونیم اونو بکشیم بیرون و از چند تیکه به یه اون تیکه کد تکراری برسیم.

```
double disabilityAmount()  
{  
    if (_seniority < 2) return 0;  
    if (_monthsDisabled > 12)  
        return 0;  
    if (_isPartTime) return 0;  
    // compute the disability amount  
}
```



```
double disabilityAmount()  
{  
    if (isNotEligableForDisability())  
        return 0;  
    // compute the disability amount  
}
```

تجزیه شرطها:

وقتی ساختارهای شرطی پیچیده داریم، بهتره که داخل IF/THEN/ELSE ها رو ساده بنویسیم.

```
if (date.before (SUMMER_START) || date.after(SUMMER_END))  
    charge = quantity * _winterRate + _winterServiceCharge;  
else  
    charge = quantity * _summerRate;
```



```
if ( notSummer(date) )  
    charge = winterCharge (quantity);  
else charge = summerCharge (quantity);
```

اگه قراره کست کردن اتفاق بیوفته، بهتره اون کست کردن رو توی بدنه تابع قرار بدیم و مقدار کست شده رو برگردونیم.

```
Object lastReading() {  
    return readings.lastElement();  
}
```

```
Reading lastReading() {  
    return (Reading) readings.lastElement();  
}
```

```
Reading lastReading =  
    (Reading) theSite.lastReading();
```



```
Reading lastReading = theSite.lastReading();
```

موانع سر راه ریفتور کردن:

مشکلات Performance: این تصور وجود داره که ریفتور کردن ممکنه اجرای محصول رو کندتر کنه.

جانیوفتادن فرهنگ ریفتور کردن: کارفرما میاد میگه آقا ما داریم پول میدیم تو فیچر جدید اضافه کنی، نه که کد رو تر تمیز کنی.

اگه داره کار میکنه، بهش دست نزن: کارفرما میاد میگه آقا به تو چه، محصول منه من میخوام کدش کثیف باشه. داره کار میکنه، بهش دست نزن.

روند توسعه همیشه تحت فشار ددلاینه: اینو میدونیم که روند توسعه همیشه تحت فشار ددلاین هاست. این وسط ریفتور کردن هم زمان میبره و ما رو به ارائه محصول قبل از ددلاین نزدیک تر نمیکنه.

ولی آیا معنی اینا اینه که ریفتور کردن خوب نیست بدرد نیخوره؟ خب نه.

معمولا 90 درصد منابع، توسط 10 درصد سیستم مصرف میشن. با ریفتور کردن ما داریم اون 10 درصد رو مشخص میکنیم و حتی برای performance هم قدم مثبت برمیداریم. نکته دیگه ای که تو بهینه سازی اهمیت داره، اینه که ما همیشه باید profiler رو روی سیستم های کد اجرا کنیم. یعنی اول باید سیستم کار رو درست انجام بده و بعد بریم سراغ بهینه سازی.

پروفایلینگ چیه؟

پروفایلینگ اندازه گیری رفتار نرم افزار موقع اجرا شونده. پروفایلینگ میتونه روی بخش های مختلفی انجام بشه و کارهای مختلفی انجام بده و اطلاعات مختلفی به ما بده. مثلا روی سرعت اجرا یا روی میزان حافظه مصرف شده و.....

نتیجه‌گیری:

با استفاده از یونیت تست، کار ما برای ریفکتور کردن راحت‌تر می‌شود.

با ریفکتور کردن، کار ما برای ایجاد تغییرات بزرگ‌تر ممکن‌تر می‌شود.

پرو فایلینگ به ما می‌گوید دقیقاً کجا و چه تغییری رو باید برای بهبود بهینه‌سازی انجام بدیم.